

Project objective

Build a **domain-agnostic** platform that:

1. Accepts an organization's configuration, historical events, and live events.
2. Learns what "normal" looks like from history.
3. Scores live events while they are still fresh.
4. Detects:
 - **(A)** one extremely abnormal event, and
 - **(B)** multiple suspicious events close together (burst).

5. Shows scores, reasons, alerts, and data freshness on a live dashboard.

Hospital and Hotel are **demo domains only**. They must not be hardcoded. A new domain is onboarded by configuration plus data mapping into the generic event schema.

Warm-data concept: detection is useful only if the dashboard reflects the live stream within the freshness SLA (section 20). Late truth is not the product.

2. Product definition

||

| - |

Item

Definition

||

| - |

Unit of work

||

| - |

EVENT (never design around "transaction")

||

| - |

Tenancy

||

| - |

Multi-organization via mandatory `organization_id`

||

| - |

Input

||

| - |

Org config + historical events + real-time events

||

| - |

Output

||

|-|

Per-event anomaly score, severity, explainable reasons, alerts, live KPIs, freshness

||

|-|

Detection modes (V1)

||

|-|

Single extreme event; suspicious-event burst

||

|-|

Not in V1

||

|-|

Sustained trends, blocking actions, domain-specific business engines

Users of the product (hackathon demo): operators watching Overview, Live Events, Anomaly Center, Anomaly Detail, and System Health.

3. Scope

In scope for the 4-week MVP:

- Generic event schema + org configuration
- Historical ingest → baselines
- Event generator (synthetic / anonymized)
- Streaming ingest (Redpanda ****or**** Kafka, not both)
- Validate, normalize, dedupe, basic enrichment / feature prep
- Unsupervised / baseline ML scoring (no DL, no LLMs)
- Combine signals into a 0–1 score; UI shows percent
- Alerts: extreme event + burst
- PostgreSQL persistence
- FastAPI REST + WebSocket
- React dashboard (pages listed in section 19)
- Freshness SLA display and breach detection
- Prometheus + Grafana
- Docker Compose local stack
- Org isolation on all queries
- Load test at hackathon scale
- Demo script in section 30

4. Non-goals

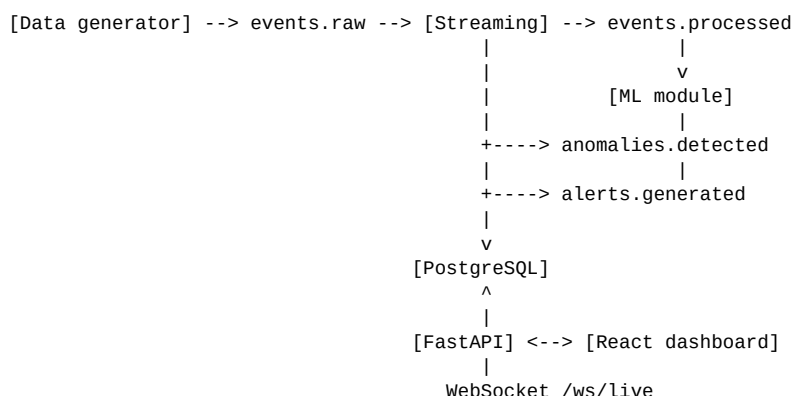
Explicitly **out of V1** (future scope only):

- Sustained-trend detection
- Autonomous blocking / auto-remediation
- Real hospital, hotel, or finance system integration
- SMS / WhatsApp / email notification providers
- LLM explanations
- Deep learning
- Kubernetes
- Cloud-scale / multi-region
- Complex SSO / enterprise IAM / RBAC (unless leftover time ****after**** MVP)
- Production compliance (HIPAA, PCI, SOC2, etc.)
- Data lake / warehouse
- Extra databases or microservices beyond the folders in section 26

...

5. Architecture

Proposed implementation detail: single Docker Compose stack; one Python streaming consumer process; one FastAPI process; one ML scoring module imported or called in-process by the streaming consumer (not a separate model server).



Prometheus scrapes streaming + backend. Grafana dashboards for ops.

Responsibility split (do not duplicate):

Layer
Owns
Does not own

Streaming
-

Consume, validate, normalize, dedupe, basic enrichment/feature prep, forward to ML, streaming metrics, produce processed/anomaly/alert topics, DLQ

||

| - |

REST, dashboard queries, WS fan-out, org CRUD UI

||

| - |

ML

||

| - |

Baseline + feature scoring + unsupervised model + combined score + reasons + severity

||

| - |

Kafka topology, HTTP API, UI

||

| - |

Backend

||

| - |

Persist what streaming/ML emit (or persist after consume of processed/anomaly/alert topics), REST, WS, org isolation, health/freshness APIs

||

| - |

Re-implement feature engineering or scoring

||

| - |

Dashboard

||

| - |

Visualize APIs + WS

||

| - |

Business logic

||

| - |

Data

||

| - |

Generators, historical normalizers, fixtures

||

| - |

Production serving

||

| - |

Monitoring

||

| - |

Metrics, Grafana, compose health, load-test harness

||

| - |

Product features

Proposed implementation detail: Backend consumes `events.processed`, `anomalies.detected`, and `alerts.generated` and writes PostgreSQL. Streaming does **not** write to PostgreSQL. This keeps module boundaries clean for parallel work.

6. Event schema

Canonical JSON for **every** event (historical, generated, live):

```
{
  "event_id": "EVT_83921",
  "organization_id": "ORG_HOSPITAL_001",
  "domain": "hospital",
  "event_type": "transaction",
  "timestamp": "2026-09-11T02:31:00Z",
  "user_id": "USR_204",
  "username": "employee_204",
  "location": "Mumbai",
  "device_id": "DEVICE_991",
  "ip_address": "192.168.1.20",
  "action": "payment",
  "status": "completed",
  "amount": 23500,
  "currency": "INR",
  "payload": {},
  "metadata": {}
}
```

Required

||

| - |

Field

Type

Rules

||

| - |

`event_id`

||

| - |

string

||

| - |

Unique per organization **proposed:** globally unique in the demo cluster; reject duplicates

||

| - |

organization_id

||

| - |

string

||

| - |

Mandatory; multi-org key

||

| - |

domain

||

| - |

string

||

| - |

Free string (hospital, hotel, or any new domain)

||

| - |

event_type

||

| - |

string

||

| - |

Domain-agnostic label (example value transaction is allowed; **do not** special-case it in code)

||

| - |

timestamp

||

| - |

string

||

| - |

UTC ISO 8601 (YYYY-MM-DDTHH:MM:SSZ or with offset converted to UTC)

Optional (may be null / omitted)

user_id, username, location, device_id, ip_address, action, status, amount, currency, payload, metadata

payload and metadata are JSON objects for domain-specific extras. Feature code **must** tolerate missing optional fields.

Validation rules (streaming)

- Missing required field → reject, prefer DLQ
- Invalid timestamp → reject, prefer DLQ
- `amount` if present must be a number
- Unknown fields: ****proposed implementation detail:**** ignore (do not fail)
- Empty `payload` / `metadata`: treat as `{}`

7. Organization configuration

Required (specified)

```
{
  "organization_id": "ORG_HOSPITAL_001",
  "organization_name": "Hospital XYZ",
  "domain": "hospital",
  "timezone": "Asia/Kolkata",
  "currency": "INR"
}
```

Extended config (eventually required; keep simple)

Proposed implementation detail: one JSON document per org, stored in `organizations.config` (JSONB). Defaults below apply if omitted.

```
{
  "organization_id": "ORG_HOSPITAL_001",
  "organization_name": "Hospital XYZ",
  "domain": "hospital",
  "timezone": "Asia/Kolkata",
  "currency": "INR",
  "baseline_lookback_days": 30,
  "anomaly_score_threshold": 0.75,
  "suspicious_score_threshold": 0.60,
  "suspicious_event_window_seconds": 300,
  "suspicious_event_count": 3,
  "freshness_sla_seconds": 5,
  "severity_bands": {
    "NORMAL": { "min": 0.00, "max": 0.39 },
    "LOW": { "min": 0.40, "max": 0.59 },
    "MEDIUM": { "min": 0.60, "max": 0.74 },
    "HIGH": { "min": 0.75, "max": 0.89 },
    "CRITICAL": { "min": 0.90, "max": 1.00 }
  }
}
```

||

|-|

Field

Meaning

||

|-|

baseline_lookback_days

||

|-|

How much history to use for user/org baselines

```

||
|-|
anomaly_score_threshold

||
|-|
is_anomaly if combined score ≥ this

||
|-|
suspicious_score_threshold

||
|-|
Event counts toward a burst if score ≥ this

||
|-|
suspicious_event_window_seconds

||
|-|
Burst window (default 5 minutes)

||
|-|
suspicious_event_count

||
|-|
Burst size (default 3)

||
|-|
freshness_sla_seconds

||
|-|
P95 event-to-dashboard target

Severity band numbers in the brief are fixed as initial defaults; orgs may override later via this config.
---
```

8. Historical data contract

Do **not** bind to a specific Kaggle file. Bind to **normalized event rows**.

Normalized requirements

Each historical row **must** map into the event schema (section 6).

Minimum historical columns after mapping:

```

||
```

| - |

Normalized field

Required

||

| - |

event_id

||

| - |

yes (synthetic IDs allowed if source has none)

||

| - |

organization_id

||

| - |

yes (assigned at load time if source has none)

||

| - |

domain

||

| - |

yes

||

| - |

event_type

||

| - |

yes

||

| - |

timestamp

||

| - |

yes

||

| - |

optional fields

||

| - |

as available

Mapping (data team)

Proposed implementation detail: data/mappings/<domain>.json describes source-column → event-field. Example:

```
{
  "domain": "hospital",
  "organization_id": "ORG_HOSPITAL_001",
  "column_map": {
    "txn_id": "event_id",
    "ts": "timestamp",
    "staff_id": "user_id",
    "city": "location",
    "amt": "amount"
  },
  "defaults": {
    "event_type": "transaction",
    "currency": "INR"
  }
}
```

Load rules

- Historical load is a **batch job** (script), not a warehouse.
- Invalid rows skipped + counted; do not abort the whole file for one bad row (**proposed**).
- After load, ML builds user-level and org/domain-level baselines.
- **Thin user history:** fall back to org/domain baselines for that user.
- **Proposed:** “thin” = fewer than 20 historical events for that `user\_id`.

Demo datasets (hospital/hotel) are fixtures under data/, still emitted as generic events.

9. Feature contract

Features are **optional per domain**. Missing inputs → feature value `null` and that feature contributes **0** to contextual score (**proposed**).

-	
Feature name	
Description	
Needs	
-	
amount	
-	
Raw amount	
-	
amount	
-	
user_amount_avg	

```

|-|
User historical mean amount
||
|-|
history + amount
||
|-|
user_amount_std
||
|-|
User historical std amount
||
|-|
history + amount
||
|-|
amount_zscore
||
|-|
(amount - avg) / std
||
|-|
avg/std
||
|-|
events_last_5min
||
|-|
Count for user in last 5 minutes
||
|-|
user_id, time
||
|-|
events_last_hour
||
|-|
Count for user in last hour
||

```

```

|-|
user_id, time
||
|-|
usual_location
||
|-|
Mode location from history
||
|-|
location
||
|-|
distance_from_usual
||
|-|
Proposed: 0 if same location string, 1 if different (no geo DB)
||
|-|
location
||
|-|
new_device
||
|-|
Device not in user history
||
|-|
device_id
||
|-|
new_ip
||
|-|
IP not in user history
||
|-|
ip_address
||

```

```

|-|
unusual_time
||
|-|
Outside user's usual hours
||
|-|
timestamp + history
||
|-|
hour_of_day
||
|-|
0-23 from UTC timestamp (proposed: also store local hour via org timezone for display)
||
|-|
timestamp
||
|-|
time_since_previous_event_seconds
||
|-|
Gap vs user's last event
||
|-|
user_id
||
|-|
user_event_frequency
||
|-|
Proposed: events / day over lookback
||
|-|
history

Streaming produces a feature-ready record (section 10). ML may add model-specific derived values but must not require extra raw fields.

---
```

10. Streaming contract

Broker choice: Redpanda or Kafka, not both.

Proposed implementation detail: Redpanda (Kafka API compatible, lighter for Docker Compose). Swap to Kafka later only if Redpanda is blocked.

Streaming owns

1. Consume events.raw
2. Validate (section 6)
3. Normalize (types, UTC timestamp, default {} objects)
4. Dedupe
5. Basic enrichment / feature prep (counts, last-seen, flags that do not need the ML model)
6. Call ML interface (section 12)
7. Produce events.processed, anomalies.detected, alerts.generated
8. Failed messages → DLQ
9. Processing metrics (section 22)

Dedup

Proposed implementation detail: key = (organization_id, event_id); in-memory + optional Redis-less TTL 1 hour in consumer; duplicate → drop, increment events_deduped_total, do not DLQ.

Feature-ready processed event (produced to `events.processed`)

```
{
  "event": { },
  "features": {
    "amount": 23500,
    "user_amount_avg": 4200.5,
    "user_amount_std": 1100.2,
    "amount_zscore": 17.5,
    "events_last_5min": 1,
    "events_last_hour": 3,
    "usual_location": "Pune",
    "distance_from_usual": 1,
    "new_device": true,
    "new_ip": true,
    "unusual_time": true,
    "hour_of_day": 2,
    "time_since_previous_event_seconds": 86000,
    "user_event_frequency": 4.2
  },
  "processing": {
    "received_at": "2026-09-11T02:31:00.120Z",
    "processed_at": "2026-09-11T02:31:00.340Z",
    "consumer_id": "streaming-0"
  }
}
```

event is the validated canonical event (section 6).

ML failure

ML exceptions must **not** crash ingestion. **Proposed:** produce processed event with features filled; skip anomaly produce; increment ml_failures_total; continue consuming.

11. Kafka/Redpanda topics

||

|-|

Topic

Key

Value

Producer

Consumer

||

|-|

events.raw

||

|-|

see below

||

|-|

canonical event

||

|-|

generator / ingest script

||

|-|

streaming

||

|-|

events.processed

||

|-|

same

||

|-|

feature-ready record

||

|-|

streaming

||

|-|

backend (+ ML already ran in streaming)

||

|-|

anomalies.detected

||

|-|

same

||

|-|

ML result (section 12)

||

|-|

streaming

||

|-|

backend

||

|-|

alerts.generated

||

|-|

same

||

|-|

alert (section 14)

||

|-|

streaming

||

|-|

backend

||

|-|

events.dlq

||

|-|

same

||

|-|

original + error

||

|-|

streaming

||

|-|

ops / replay script

Message key (fixed): organization_id:user_id if user_id present, else organization_id.

Proposed implementation detail:

- Replication factor 1, partitions 3 in Compose
- Retention 24h for `events.raw` / processed; 72h for anomalies/alerts/DLQ
- JSON encoding, no Schema Registry in V1
- DLQ envelope:

```
{
  "original_topic": "events.raw",
  "error_code": "VALIDATION_ERROR",
  "error_message": "missing timestamp",
  "failed_at": "2026-09-11T02:31:00.200Z",
  "payload": {}
}
```

12. ML contract

Practical and explainable. **No deep learning. No LLMs.**

Proposed implementation detail: combine:

1. Historical baseline / z-score style signals
2. Contextual feature rules (new device, unusual time, burst counts, etc.)
3. Isolation Forest (or similar unsupervised sklearn model) on available numeric features
4. Weighted blend → anomaly_score in [0, 1]

Exact model class and weights are **not frozen**; the **interface is frozen**.

Module interface (Python, conceptual)

```
score_event(event: Event, features: FeatureDict, org_config: OrgConfig, baselines: Baselines) -> AnomalyResult
fit_baselines(historical_events: Iterable[Event], org_config: OrgConfig) -> Baselines
fit_model(feature_matrix, org_config) -> ModelArtifact
```

ml/ exposes these functions. Streaming calls score_event. Training is a batch script in ml/.

Output (every scored event)

```
{
  "event_id": "EVT_83921",
  "organization_id": "ORG_HOSPITAL_001",
  "anomaly_score": 0.96,
  "risk_score": 0.96,
  "severity": "CRITICAL",
  "is_anomaly": true,
  "reasons": [
    {
      "code": "AMOUNT_ZSCORE",
      "feature": "amount_zscore",
      "detail": "Amount 23500 is 17.5 std above user mean 4200",
      "contribution": 0.42
    },
    {
      "code": "NEW_DEVICE",
      "feature": "new_device",
      "detail": "Device DEVICE_991 not seen for this user",
      "contribution": 0.18
    }
  ],
  "model_version": "baseline-iforest-v1",
  "processed_at": "2026-09-11T02:31:00.400Z"
}
```

||

```

|-|
Field
Rules
||
|-|
anomaly_score
||
|-|
float 0–1 inclusive
||
|-|
risk_score
||
|-|
Proposed: same as anomaly_score in V1 (field reserved if later split)
||
|-|
severity
||
|-|
from bands (section 13)
||
|-|
is_anomaly
||
|-|
anomaly_score >= org.anomaly_score_threshold
||
|-|
reasons
||
|-|
from feature/rule signals only
||
|-|
model_version
||
|-|
string set at train/load time

```

UI displays anomaly_score as percent, e.g. 96%.

13. Anomaly scoring

Internal score

- Range `[0.00, 1.00]`
- Clamp if a component exceeds bounds

Severity (fixed initial bands; org-configurable later)

||

| - |

Severity

Score range

||

| - |

NORMAL

||

| - |

0.00–0.39

||

| - |

LOW

||

| - |

0.40–0.59

||

| - |

MEDIUM

||

| - |

0.60–0.74

||

| - |

HIGH

||

| - |

0.75–0.89

||

| - |

CRITICAL

||

|-|

0.90–1.00

Inclusive on both ends except **proposed**: a value on a boundary uses the **higher** severity (e.g. 0.40 → LOW). 1.00 → CRITICAL.

Blend (**proposed implementation detail** — needs team/ML approval)

$\text{anomaly_score} = 0.40 * \text{baseline_score} + 0.30 * \text{contextual_score} + 0.30 * \text{iforest_score}$

- ``baseline_score``: squashed amount z-score and frequency deviation (0 if features missing)
- ``contextual_score``: fraction of fired rules (new device, new IP, unusual time, location change, etc.)
- ``iforest_score``: Isolation Forest decision mapped to 0–1

If Isolation Forest is not yet trained: `iforest_score = 0` and `model_version` notes baseline-only.

Detection mode A — single extreme event

Alert when `severity == CRITICAL` or `anomaly_score >= 0.90` (same band) or **proposed**: also when `is_anomaly` and severity is HIGH+ depending on org flag. **Fixed for demo**: fire `SINGLE_EXTREME_EVENT` when `score ≥ 0.90`.

Detection mode B — suspicious burst

Alert when `≥ suspicious_event_count` events from the **same** `user_id` with `score ≥ suspicious_score_threshold` inside `suspicious_event_window_seconds`.

Default: **3+ suspicious in 5 minutes**, same user.

No `user_id` → **proposed**: burst detection skipped for that event (extreme-event path still applies).

Not in V1: sustained trends.

14. Alert contract

```
{
  "alert_id": "ALT_10001",
  "organization_id": "ORG_HOSPITAL_001",
  "alert_type": "SINGLE_EXTREME_EVENT",
  "severity": "CRITICAL",
  "user_id": "USR_204",
  "event_ids": ["EVT_83921"],
  "window_start": "2026-09-11T02:31:00Z",
  "window_end": "2026-09-11T02:31:00Z",
  "title": "Critical anomaly on event EVT_83921",
  "summary": "Single event scored 0.96 (CRITICAL)",
  "reasons": [],
  "created_at": "2026-09-11T02:31:00.410Z"
}
```

`alert_type` enum:

- ``SINGLE_EXTREME_EVENT``
- ``SUSPICIOUS_EVENT_BURST``

`reasons` copied from the triggering event(s) feature/rule explanations.

Proposed: `alert_id = ALT_ + ulid/uuid hex`. Dedup burst alerts: one burst alert per user per window; do not emit a new burst alert for every additional event in the same open window (update `event_ids` or emit once when threshold first crossed — **choose: emit once when threshold first crossed**, simplest).

15. Explainability

- Reasons come **only** from feature and rule signals.
- No LLM text generation.
- Each reason has `code`, `feature`, human-readable `detail`, optional `contribution`.
- Dashboard Anomaly Detail lists reasons as received; frontend does not invent copy.
- **Proposed** standard codes: `AMOUNT\_ZSCORE`, `HIGH\_VELOCITY\_5MIN`, `HIGH\_VELOCITY\_HOUR`, `LOCATION\_CHANGE`, `NEW\_DEVICE`, `NEW\_IP`, `UNUSUAL\_TIME`, `SHORT\_INTERVAL`, `IFOREST\_OUTLIER`, `THIN\_USER\_BASELINE\_FALLBACK`.

16. Backend API contract

Stack: **Python, FastAPI, Pydantic, SQLAlchemy, PostgreSQL, WebSockets, Pytest.**

Base path: `/api`

Proposed: JSON only. `organization_id` **required** query param on all org-scoped GETs except `/api/health`.

Missing org → 400.

Proposed error envelope:

```
{
  "error": {
    "code": "NOT_FOUND",
    "message": "Anomaly not found"
  }
}
```

Auth: **simplified**. **Proposed implementation detail:** no login in MVP; optional `X-Organization-Id` header mirroring query param for dashboard convenience. No SSO/RBAC unless time after MVP.

GET `/api/health`

200

```
{
  "status": "ok",
  "backend": "ok",
  "database": "ok",
  "broker": "ok",
  "streaming": "ok",
  "ml": "ok",
  "checked_at": "2026-09-11T02:31:05Z"
}
```

status is ok only if all components are ok; else degraded. Component values: ok | degraded | down | unknown.

GET `/api/organizations`

200

```
{
  "organizations": [
    {
      "organization_id": "ORG_HOSPITAL_001",
      "organization_name": "Hospital XYZ",
      "domain": "hospital",
      "timezone": "Asia/Kolkata",
      "currency": "INR"
    }
  ]
}
```

GET `/api/dashboard?organization_id=`

Overview KPIs. **Proposed** extras only as needed for the Overview page.

200

```
{
  "organization_id": "ORG_HOSPITAL_001",
  "events_per_minute": 42.0,
  "events_total": 12890,
  "anomalies_total": 37,
  "critical_total": 4,
  "anomaly_score_avg": 0.12,
  "freshness_seconds": 2.3,
  "freshness_status": "GREEN",
  "pipeline_status": "ok",
  "window_seconds": 60
}
```

freshness_status: GREEN | WARNING | CRITICAL (section 20).

events_per_minute: **proposed** over last 60 seconds.

GET `/api/events?organization_id=&limit=&cursor=&user_id=`

Newest first. **Proposed:** limit default 50, max 200. Cursor opaque.

200

```
{
  "items": [
    {
      "event_id": "EVT_83921",
      "organization_id": "ORG_HOSPITAL_001",
      "domain": "hospital",
      "event_type": "transaction",
      "timestamp": "2026-09-11T02:31:00Z",
      "user_id": "USR_204",
      "username": "employee_204",
      "location": "Mumbai",
      "device_id": "DEVICE_991",
      "ip_address": "192.168.1.20",
      "action": "payment",
      "status": "completed",
      "amount": 23500,
      "currency": "INR",
      "anomaly_score": 0.96,
      "severity": "CRITICAL",
      "is_anomaly": true
    }
  ],
  "next_cursor": null
}
```

GET `/api/anomalies?organization_id=&limit=&cursor=&severity=&user_id=`

Anomalous events only (`is_anomaly = true`). Newest first.

200

```
{
  "items": [
    {
      "event_id": "EVT_83921",
      "organization_id": "ORG_HOSPITAL_001",
      "timestamp": "2026-09-11T02:31:00Z",
      "user_id": "USR_204",
      "anomaly_score": 0.96,
      "severity": "CRITICAL",
      "is_anomaly": true,
      "model_version": "baseline-iforest-v1",
      "processed_at": "2026-09-11T02:31:00.400Z"
    }
  ],
  "next_cursor": null
}
```

GET `/api/anomalies/{event_id}?organization_id=`

200 — event + score + reasons + linked alerts

```
{
  "event_id": "EVT_83921",
  "organization_id": "ORG_HOSPITAL_001",
  "event": {},
  "features": {},
  "anomaly_score": 0.96,
  "risk_score": 0.96,
  "severity": "CRITICAL",
  "is_anomaly": true,
  "reasons": [],
  "model_version": "baseline-forest-v1",
  "processed_at": "2026-09-11T02:31:00.400Z",
  "alerts": [
    {
      "alert_id": "ALT_10001",
      "alert_type": "SINGLE_EXTREME_EVENT",
      "severity": "CRITICAL"
    }
  ]
}
```

404 if event missing **or** belongs to another org.

GET `/api/alerts?organization_id=&limit=&cursor=&alert_type=`

200

```
{
  "items": [
    {
      "alert_id": "ALT_10001",
      "organization_id": "ORG_HOSPITAL_001",
      "alert_type": "SINGLE_EXTREME_EVENT",
      "severity": "CRITICAL",
      "user_id": "USR_204",
      "event_ids": ["EVT_83921"],
      "title": "Critical anomaly on event EVT_83921",
      "summary": "Single event scored 0.96 (CRITICAL)",
      "created_at": "2026-09-11T02:31:00.410Z"
    }
  ],
  "next_cursor": null
}
```

GET `/api/pipeline/freshness?organization_id=`

200

```
{
  "organization_id": "ORG_HOSPITAL_001",
  "freshness_seconds": 2.3,
  "freshness_p95_seconds": 3.1,
  "freshness_status": "GREEN",
  "sla_seconds": 5,
  "last_event_timestamp": "2026-09-11T02:31:00Z",
  "last_dashboard_visible_at": "2026-09-11T02:31:02.300Z",
  "pipeline_status": "ok"
}
```

No additional REST endpoints in V1.

17. WebSocket contract

URL: WS `/ws/live?organization_id=`

Subscribe to messages the dashboard needs. **Do not** stream unrelated types.

Proposed: server sends JSON text frames. Client does not send (listen-only) except optional ping.

Server → client envelope

```
{
  "type": "anomaly",
  "organization_id": "ORG_HOSPITAL_001",
  "emitted_at": "2026-09-11T02:31:00.450Z",
  "payload": {}
}
```

Types (only these)

||

| - |

type

When

payload

||

| - |

event

||

| - |

New processed event persisted

||

| - |

same shape as an item in GET /api/events

||

| - |

anomaly

||

| - |

is_anomaly true

||

| - |

same shape as anomaly list item **plus** reasons

||

| - |

alert

||

| - |

New alert

||

| - |

same shape as alert list item

||

| - |

dashboard

||

|-|

KPI snapshot

||

|-|

same shape as GET /api/dashboard

||

|-|

freshness

||

|-|

Freshness update

||

|-|

same shape as GET /api/pipeline/freshness

||

|-|

health

||

|-|

Component status change

||

|-|

same shape as GET /api/health

Example type: anomaly:

```
{
  "type": "anomaly",
  "organization_id": "ORG_HOSPITAL_001",
  "emitted_at": "2026-09-11T02:31:00.450Z",
  "payload": {
    "event_id": "EVT_83921",
    "organization_id": "ORG_HOSPITAL_001",
    "timestamp": "2026-09-11T02:31:00Z",
    "user_id": "USR_204",
    "anomaly_score": 0.96,
    "severity": "CRITICAL",
    "is_anomaly": true,
    "reasons": [],
    "model_version": "baseline-iforest-v1",
    "processed_at": "2026-09-11T02:31:00.400Z"
  }
}
```

On disconnect, dashboard reconnects with backoff (section 21) and refreshes via REST.

18. Database schema

PostgreSQL only. All queries **must** filter `organization_id` (except global health). Extra tables only if necessary; V1 tables:

``organizations``

```
||
|-|
Column
Type
Notes
||
|-|
organization_id
||
|-|
TEXT PK
||
|-|
||
|-|
organization_name
||
|-|
TEXT NOT NULL
||
|-|
||
|-|
domain
||
|-|
TEXT NOT NULL
||
|-|
||
|-|
timezone
||
|-|
TEXT NOT NULL
||
```

```

|-|
||
|-|
currency
||
|-|
TEXT
||
|-|
||
|-|
config
||
|-|
JSONB NOT NULL
||
|-|
extended config (section 7)
||
|-|
created_at
||
|-|
TIMESTAMPTZ
||
|-|
proposed default now( )

`events`
||
|-|
Column
Type
Notes
||
|-|
organization_id
||
|-|
TEXT NOT NULL

```

```
||
|-|
FK orgs
||
|-|
event_id
||
|-|
TEXT NOT NULL
||
|-|
||
|-|
domain
||
|-|
TEXT NOT NULL
||
|-|
||
|-|
event_type
||
|-|
TEXT NOT NULL
||
|-|
||
|-|
timestamp
||
|-|
TIMESTAMPTZ NOT NULL
||
|-|
||
|-|
user_id
||
```

| - |
TEXT
-
username
-
TEXT
-
-
location
-
TEXT
-
-
device_id
-
TEXT
-
-
ip_address
-
TEXT
-
-
action

```
||
|-|
TEXT
||
|-|
||
|-|
status
||
|-|
TEXT
||
|-|
||
|-|
amount
||
|-|
NUMERIC
||
|-|
||
|-|
currency
||
|-|
TEXT
||
|-|
||
|-|
payload
||
|-|
JSONB
||
|-|
||
|-|
```

metadata

||

|-|

JSONB

||

|-|

||

|-|

received_at

||

|-|

TIMESTAMPTZ

||

|-|

||

|-|

ingested_at

||

|-|

TIMESTAMPTZ

||

|-|

when backend wrote the row

PK: (organization_id, event_id)

Index (proposed): (organization_id, timestamp DESC), (organization_id, user_id, timestamp DESC)

`anomaly\_results`

||

|-|

Column

Type

Notes

||

|-|

organization_id

||

|-|

TEXT NOT NULL

||

|-|

```
||
|-|
event_id
||
|-|
TEXT NOT NULL
||
|-|
||
|-|
anomaly_score
||
|-|
DOUBLE PRECISION NOT NULL
||
|-|
||
|-|
risk_score
||
|-|
DOUBLE PRECISION
||
|-|
||
|-|
severity
||
|-|
TEXT NOT NULL
||
|-|
||
|-|
is_anomaly
||
|-|
BOOLEAN NOT NULL
||
```

```

|-|
||
|-|
reasons
||
|-|
JSONB NOT NULL
||
|-|
||
|-|
features
||
|-|
JSONB
||
|-|
snapshot for detail page
||
|-|
model_version
||
|-|
TEXT NOT NULL
||
|-|
||
|-|
processed_at
||
|-|
TIMESTAMPTZ NOT NULL
||
|-|
PK: (organization_id, event_id)
Index (proposed): (organization_id, is_anomaly, processed_at DESC)

`alerts`
||
|-|

```

Column

Type

Notes

||

|-|

alert_id

||

|-|

TEXT PK

||

|-|

||

|-|

organization_id

||

|-|

TEXT NOT NULL

||

|-|

||

|-|

alert_type

||

|-|

TEXT NOT NULL

||

|-|

||

|-|

severity

||

|-|

TEXT NOT NULL

||

|-|

||

|-|

user_id

||

```
| - |  
TEXT  
||  
| - |  
||  
| - |  
event_ids  
||  
| - |  
JSONB NOT NULL  
||  
| - |  
array of strings  
||  
| - |  
window_start  
||  
| - |  
TIMESTAMPTZ  
||  
| - |  
||  
| - |  
window_end  
||  
| - |  
TIMESTAMPTZ  
||  
| - |  
||  
| - |  
title  
||  
| - |  
TEXT NOT NULL  
||  
| - |  
||  
| - |
```

summary

||

|-|

TEXT NOT NULL

||

|-|

||

|-|

reasons

||

|-|

JSONB

||

|-|

||

|-|

created_at

||

|-|

TIMESTAMPTZ NOT NULL

||

|-|

Index (proposed): (organization_id, created_at DESC)

`pipeline\_health`

||

|-|

Column

Type

Notes

||

|-|

id

||

|-|

BIGSERIAL PK

||

|-|

proposed

||

```
| - |
organization_id
||
| - |
TEXT
||
| - |
null = cluster-wide
||
| - |
component
||
| - |
TEXT NOT NULL
||
| - |
streaming | ml | backend | db | broker | websocket
||
| - |
status
||
| - |
TEXT NOT NULL
||
| - |
ok | degraded | down
||
| - |
consumer_lag
||
| - |
BIGINT
||
| - |
||
| - |
freshness_seconds
||
| - |
```

DOUBLE PRECISION

||

|-|

||

|-|

latency_p95_seconds

||

|-|

DOUBLE PRECISION

||

|-|

||

|-|

detail

||

|-|

TEXT

||

|-|

||

|-|

observed_at

||

|-|

TIMESTAMPTZ NOT NULL

||

|-|

Proposed: keep latest row per (organization_id, component); workers upsert.

No extra databases.

19. Dashboard requirements

Stack: **React, Vite, TypeScript, Tailwind, Recharts, Axios, WebSocket, Lucide.**

Pages (exactly these; no extra pages):

1. **Overview** — events/min, totals, anomalies, critical count, average score, freshness (Data freshness: 2.3s), pipeline status.
2. **Live Events** — streaming table of recent events; highlight anomalies.
3. **Anomaly Center** — list of anomalies / alerts.
4. **Anomaly Detail** — one event: score %, severity, reasons, raw event, features.
5. **System Health** — streaming / ML / backend / DB status, consumer lag, latency, freshness.

Empty, loading, and error states required on every page.

Org selector uses GET /api/organizations.

Live updates via /ws/live; REST as fallback.

Copy must be real (no lorem). Hospital/Hotel appear only as demo org names/data.

20. Freshness SLA

Targets (fixed)

- **P95 event-to-dashboard $\leq 5s$** under demo load
- **Critical detection E2E $\leq 5s$** (event timestamp/receipt $\rightarrow$ visible anomaly/alert)
- Display: `Data freshness: 2.3s` (one decimal **proposed**)

Status colors (fixed)

-
Status
Freshness
-
GREEN
-
< 5s
-
WARNING
-
5–10s
-
CRITICAL
-
> 10s

SLA threshold is **configurable** per org (freshness_sla_seconds, default 5). Colors still use 5s / 10s bands unless org overrides **proposed** freshness_warning_seconds (default 5) and freshness_critical_seconds (default 10).

How freshness is calculated (**proposed implementation detail**)

For each processed event:

```
event_to_dashboard_ms = dashboard_visible_at - event_received_at
```

- ``event_received_at``: set by streaming when the message is read from ``events.raw`` (wall clock UTC).
- ``dashboard_visible_at``: set by backend when the row is committed ****and**** a WS ``event`/`anomaly`` frame is emitted (use emit time). If WS is down, use DB commit time (REST pollers).

Displayed freshness: P95 of `event_to_dashboard` over the last **60 seconds** for the selected org. If no events in window, show last known value and mark pipeline stale (section 21).

Critical detection E2E: same formula restricted to events with `severity == CRITICAL`.

Backend exposes both latest sample and P95 on `/api/pipeline/freshness`.

21. Failure/recovery behaviour

Hackathon-simple. **No silent loss.** Detect failures and SLA breaches.

||

| - |

Failure

Detection

Behaviour

||

| - |

Broker down

||

| - |

health check / produce errors

||

| - |

Generator retries; buffer locally **proposed:** in-memory queue max 10k then drop oldest + metric `events_dropped_total`. Dashboard: broker down.

||

| - |

Consumer down

||

| - |

lag grows; no `processed_at`

||

| - |

Backend freshness CRITICAL; System Health `streaming=down`. On restart, consume from last committed offset. Redpanda retention is the buffer.

||

| - |

ML fail

||

| - |

exception in score_event

||

|-|

Do not crash consumer. Metrics + skip anomaly. Optionally DLQ with ML_ERROR.

||

|-|

Backend down

||

|-|

health; WS drop

||

|-|

Dashboard error state; streaming continues; topics retain. On recovery, consume backlog.

||

|-|

DB down

||

|-|

SQL errors

||

|-|

Backend degraded; do not ack backend topic offsets (**proposed** so replay is possible). Retry with backoff.

||

|-|

WS disconnect

||

|-|

client on close

||

|-|

Reconnect 1s, 2s, 4s, max 15s **proposed**; REST refetch.

||

|-|

Stale stream

||

|-|

no events while generator should be on; freshness > 10s

||

|-|

pipeline_status=stale; freshness CRITICAL; Health page warning.

||

|-|

Invalid event

||

|-|

validation

||

|-|

Reject; increment failed; **prefer DLQ**; never write as valid event.

Retries: **proposed** 3 attempts, exponential backoff $0.2s \times 2^n$, then DLQ / drop with metric.

Invalid $\neq$ crash.

22. Monitoring

Prometheus + Grafana. Keshav owns dashboards and exporters.

Metrics (minimum)

||

|-|

Metric

Type

Source

||

|-|

events_received_total

||

|-|

counter

||

|-|

streaming

||

|-|

events_processed_total

||

|-|

counter

||

|-|

```
streaming
||
|-|
events_failed_total
||
|-|
counter
||
|-|
streaming
||
|-|
events_deduped_total
||
|-|
counter
||
|-|
streaming
||
|-|
events_per_second
||
|-|
gauge or rate()
||
|-|
streaming
||
|-|
processing_latency_seconds
||
|-|
histogram
||
|-|
streaming (receive → processed)
||
|-|
```

processing_latency_p95_seconds

||

|-|

gauge or histogram quantile

||

|-|

streaming

||

|-|

anomalies_total

||

|-|

counter

||

|-|

streaming/ML

||

|-|

critical_anomalies_total

||

|-|

counter

||

|-|

streaming/ML

||

|-|

consumer_lag

||

|-|

gauge

||

|-|

streaming / kafka exporter

||

|-|

freshness_seconds

||

|-|

```

gauge
||
|-|
backend
||
|-|
freshness_p95_seconds
||
|-|
gauge
||
|-|
backend
||
|-|
service_health
||
|-|
gauge 1/0
||
|-|
each service

```

Proposed: scrape interval 5s for demo. Grafana boards: Pipeline, Freshness, Errors. No extra observability stack (no ELK).

23. Load testing

Proposed implementation detail: **Locust** in monitoring/ or data/ load folder — prefer monitoring/locustfile.py so Keshav owns it; generator remains Akshara's. If Locust proves impractical in week 3, fall back to the existing generator at higher rate **and document** the substitution.

Hackathon-scale targets (****proposed**** — approve in week 3)

```

||
|-|
Measure
Target
||
|-|
Sustained ingest
||
|-|

```

50 events/s for 5 minutes, 1 org

||

|-|

Burst

||

|-|

200 events/s for 30 seconds

||

|-|

API GET /api/dashboard

||

|-|

p95 < 200 ms

||

|-|

Event processing latency

||

|-|

p95 < 2 s

||

|-|

Dashboard freshness

||

|-|

p95 < 5 s

||

|-|

HTTP error rate

||

|-|

< 1%

||

|-|

Consumer lag

||

|-|

< 1000 messages after burst settles (60s)

Report: throughput, API latency, event processing latency, dashboard freshness, error rate, consumer lag.

24. Security baseline

- All reads/writes scoped by `organization\_id` (no cross-org leak in list/detail).
- No secrets in git. Env vars. Ship `.env.example` only.
- Validate every event before persist/score.
- **Synthetic or anonymized data only** (no real patient/guest/finance data).
- Auth simplified (section 16). No SSO/RBAC unless leftover time after MVP.
- **Proposed:** bind Compose ports to localhost; default DB password in `.env.example` only for local demo.

25. Team ownership

Kavisha — Backend + System Architecture

- `backend/` FastAPI, Pydantic models, SQLAlchemy, migrations, REST, WS, org isolation
- Architecture consistency with this contract
- Coordinates interface freezes
- **Not:** Redpanda topology, sklearn training, React pages

Ashish — Streaming + Data Engineering

- `streaming/` Redpanda/Kafka client, producers/consumers, ingest, validation, normalization, dedupe, stream processing, feature-ready output, streaming metrics
- Stable contracts toward ML (`score\_event` input) and backend (topics + JSON)
- **Not:** whole backend, **not** ML model design/training

Divay — Data Science / ML

- `ml/` baselines, Isolation Forest (or similar), blend, reasons, `model\_version`, train scripts
- Implements `score\_event` / `fit\_\*` against frozen feature dict
- **Not:** HTTP API, UI, broker ops

Avni — Frontend / Dashboard

- `dashboard/` five pages, Tailwind, Recharts, Axios, WS, Lucide
- Consumes API/WS only; no scoring logic
- **Not:** FastAPI, streaming

Akshara — Data Generation + Data Pipeline Support

- `data/` generators, historical mapping/normalization, demo scenarios (normal, extreme, burst)
- Helps Ashish with sample payloads and replay files
- **Not:** production consumer, not ML internals

Keshav — DevOps + Monitoring / Reliability

- `docker-compose.yml`, Prometheus, Grafana, health wiring, load test, freshness/failure demo (stop a service)
- `.env.example`, local run docs support
- **Not:** product feature pages, model math

Parallel work: freeze this file → mock JSON fixtures in each module → integrate.

Dev order (fixed): freeze contracts → repo structure → skeleton → vertical E2E slice → ML → enterprise extras → test/polish.

26. Development dependencies

Repository layout (fixed; no extra folders)

```
real-time-anomaly-platform/  
  backend/  
  streaming/  
  ml/  
  dashboard/  
  data/  
  monitoring/  
  docs/  
  docker-compose.yml  
  PROJECT_RULES.md  
  CONTRACT.md  
  README.md
```

Proposed implementation detail: this git repository is `real-time-anomaly-platform` (folders at repo root), not a nested extra wrapper directory.

Allowed stack

Python, FastAPI, PostgreSQL, Redpanda (or Kafka), React, sklearn, Prometheus, Grafana, Docker Compose.

Avoid

Kubernetes, Spark, Airflow, TensorFlow, PyTorch, RabbitMQ, Elasticsearch, complex cloud, multi-DB, microservice explosion, second UI component library.

Runtime processes in Compose (**proposed**)

redpanda, postgres, streaming, backend, dashboard, prometheus, grafana, optional generator.

27. Git workflow

||

| - |

Branch

Policy

||

| - |

main

||

| - |

Protected; no direct push

||

| - |

develop

||

| - |

Integration branch

||

|-|

feature/backend

||

|-|

Kavisha

||

|-|

feature/streaming

||

|-|

Ashish

||

|-|

feature/ml

||

|-|

Divay

||

|-|

feature/dashboard

||

|-|

Avni

||

|-|

feature/data

||

|-|

Akshara

||

|-|

feature/monitoring

||

|-|

Keshav

- PRs into `develop`, then `develop` → `main`

- Small commits

- Contract changes: PR that the whole team sees

28. Definition of Done

A feature is done when:

1. Matches this contract (or an approved contract PR)
2. Scoped to the owner's module
3. Org isolation preserved
4. Failure path: no silent drop (metric, DLQ, or visible health)
5. Pytest / frontend build as applicable
6. Demo path not broken
7. README / docs / updated only if run steps change

29. MVP

End-to-end slice, in order:

historical → **baseline** → **event generator** → **Redpanda** → **stream processing** → **anomaly detection** → **PostgreSQL**
→ **FastAPI** → **WebSocket** → **React**

Must show: score, explanation, live KPI, live alert, freshness.

Hospital/Hotel are interchangeable demo configs, not engines.

30. Demo flow

Fourteen steps (fixed):

1. Start the stack.
2. Show **normal** events flowing.
3. Show the **live dashboard**.
4. Show **freshness** (Data freshness: Xs, GREEN).
5. Inject **one highly abnormal** event.
6. **ML detects** it (score + severity).
7. Dashboard **auto-updates** (WS).
8. Show **alert with reasons**.
9. Generate **multiple suspicious** events close together.
10. Show **burst** alert.
11. **Stop a service** (e.g. streaming or backend).
12. Show **freshness/health failure**.
13. **Recover** the service.
14. Show **processing again**. Close on the warm-data concept (detection while data is still fresh).

31. Four-week plan

Week 1 — contracts, skeleton, E2E slice

- Freeze `CONTRACT.md` / `PROJECT\_RULES.md`
- Repo structure + Docker Compose (Redpanda, Postgres, empty services)
- Generator emitting valid events
- Streaming consume/validate/produce
- Backend + dashboard skeleton
- Vertical slice: event appears on Live Events (score can be stub)

Week 2 — detection

- Historical load + baselines
- Real scoring + severity
- Explanations
- Live alerts (extreme + burst)

Week 3 — reliability

- Monitoring (Prometheus/Grafana)
- Freshness SLA wiring and colors
- Failure handling (DLQ, health, WS reconnect, stale stream)
- Org isolation verification
- Load testing

Week 4 — ship the demo

- Integration, UI polish, testing, demo rehearsal, docs, PPT
- ****No new architecture in week 4****

32. Acceptance criteria

The project is accepted for demo if all of the following hold:

1. A new domain can be added with org config + column mapping only (no hospital/hotel hardcoded logic).
2. Events follow section 6; required fields enforced; optional fields may be absent.
3. Historical data is generic events; thin users use org/domain baselines.
4. Live scoring produces `anomaly_score` 0–1, severity band, `is_anomaly`, reasons, `model_version`.
5. Single extreme event (score ≥ 0.90) creates `SINGLE_EXTREME_EVENT` and UI alert with reasons.
6. Configurable burst (default 3 suspicious in 5 minutes, same user) creates `SUSPICIOUS_EVENT_BURST`.
7. No sustained-trend feature exists.
8. Topics `events.raw`, `events.processed`, `anomalies.detected`, `alerts.generated` exist; DLQ used for invalids.
9. Message key is `organization_id:user_id` or `organization_id`.
10. All listed REST endpoints and `WS /ws/live` match section 16–17.
11. PostgreSQL has organizations, events, `anomaly_results`, alerts, `pipeline_health`; queries filter `organization_id`.
12. Five dashboard pages exist and no others; Overview shows freshness and KPIs.
13. Under demo load, P95 freshness $\leq 5s$ and critical path $\leq 5s$; status colors GREEN/WARNING/CRITICAL.

14. Stopping a service is visible on System Health; freshness degrades; recovery resumes processing; invalid events go to DLQ.
15. Prometheus metrics listed in section 22 are scrapeable; Grafana shows them.
16. Load test produces the measurements in section 23.
17. No secrets in git; synthetic data only; org isolation holds.
18. Folder layout and stack match section 26; no forbidden tech.
19. Demo can be executed as the 14 steps in section 30.
20. Work landed via the git workflow in section 27.

Appendix A — Proposed vs frozen (quick index)